

Array Multidimensionali

Array Multidimensionali

Come usare array a due o più dimensioni in C++.

Array con più dimensioni

Un array normale è come una fila di scatole. Ma a volte i dati sono naturalmente organizzati in una **griglia**: i voti di più studenti in più materie, le celle di una scacchiera, i pixel di un'immagine.

Un **array bidimensionale** è come una tabella con righe e colonne: hai bisogno di due numeri per identificare ogni cella.

Dichiarare un array 2D

```
tipo nome[righe][colonne];
```

```
int tabella[3][4]; // 3 righe, 4 colonne – 12 celle in totale
```

Inizializzare un array 2D

Puoi inizializzarlo riga per riga, usando parentesi graffe interne:

```
int matrice[2][3] = {  
    {1, 2, 3}, // riga 0  
    {4, 5, 6}  // riga 1  
};
```

Accedere agli elementi

Devi usare **due indici**: `[riga] [colonna]` . Entrambi partono da 0.

```
int matrice[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

cout << matrice[0][0] << endl; // 1 - riga 0, colonna 0
cout << matrice[0][2] << endl; // 3 - riga 0, colonna 2
cout << matrice[1][1] << endl; // 5 - riga 1, colonna 1
```

Immagina le coordinate come `[Y] [X]` : prima la riga (verticale), poi la colonna (orizzontale).

Scorrere con due cicli annidati

Per visitare tutti gli elementi di una matrice, si usano due cicli `for` annidati: uno per le righe, uno per le colonne:

```
int matrice[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int i = 0; i < 3; i++) { // ciclo esterno: scorre le righe
    for (int j = 0; j < 3; j++) { // ciclo interno: scorre le colonne
        della riga i
        cout << matrice[i][j] << "\t";
    }
    cout << endl; // vai a capo dopo ogni riga
}
```

Output:

```
1  2  3
4  5  6
7  8  9
```

Esempio: registro voti degli studenti

Un uso pratico: memorizzare i voti di più studenti in più materie.

```
#include <iostream>
using namespace std;

int main() {
    const int STUDENTI = 3;
    const int MATERIE = 4;

    // voti[studente][materia]
    int voti[STUDENTI][MATERIE] = {
        {8, 7, 9, 6},    // Alice
        {5, 8, 7, 9},    // Bob
        {9, 9, 8, 10}    // Carlo
    };

    string nomi[] = {"Alice", "Bob", "Carlo"};
    string materie[] = {"Matematica", "Italiano", "Inglese", "Storia"};

    // Stampa la tabella con le medie
    for (int s = 0; s < STUDENTI; s++) {
        cout << nomi[s] << ": ";
        int somma = 0;
        for (int m = 0; m < MATERIE; m++) {
            cout << voti[s][m] << " ";
            somma += voti[s][m];
        }
        double media = (double)somma / MATERIE;
        cout << "→ media: " << media << endl;
    }

    return 0;
}
```

Esempio: scacchiera

```
#include <iostream>
using namespace std;

int main() {
    char scacchiera[8][8];

    // Riempiamo la scacchiera alternando # e .
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            // Se la somma degli indici è pari, casella nera
            if ((i + j) % 2 == 0) {
                scacchiera[i][j] = '#';
            } else {
                scacchiera[i][j] = '.';
            }
        }
    }

    // Stampa la scacchiera
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            cout << scacchiera[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Array a tre dimensioni

Puoi avere anche array con tre o più dimensioni, anche se diventano difficili da visualizzare:

```
int cubo[2][3][4]; // 2 "strati", 3 righe, 4 colonne

// Accesso con tre indici
cubo[0][1][2] = 42;
```

Un array 3D è come un cubo: ha profondità, altezza e larghezza. Nella pratica, gli array 2D coprono la maggior parte dei casi.

Limitazioni degli array C++

Gli array tradizionali hanno alcuni svantaggi:

- La dimensione deve essere fissa e nota prima dell'esecuzione
- Non si "ricordano" quanti elementi contengono
- Non controllano se stai uscendo dai limiti

Per questi motivi, nelle applicazioni moderne si usa spesso `vector` (vedi la sezione apposita), che è più flessibile e sicuro.